

Lecture 12 & 13 - Module 4.4 Function Convexity and Gradient Descent

COMP 551 Applied machine learning

Yue Li
Assistant Professor
School of Computer Science
McGill University

Feb 13 & 20, 2025

Outline

Objectives

General gradient descent algorithm

Function convexity

Choice of learning rate

Stochastic Gradient Descent (SGD)

Summary

Learning objectives

Understanding the basic ideas of

- Function convexity
- Gradient descent as general algorithm (reinforcing the concepts learned in Module 4.2 and 4.3)
- Stochastic gradient descent and method of momentum
- Adaptive learning rate

Outline

Objectives

General gradient descent algorithm

Function convexity

Choice of learning rate

Stochastic Gradient Descent (SGD)

Summary

Numerical optimization is the workhorse of machine learning algorithms

In this module, we consider:

- Unconstrained
- Convexity testing
- Local optima
- Smooth error surface
- First-order gradient

In Module 4.5 Regularization, we will consider:

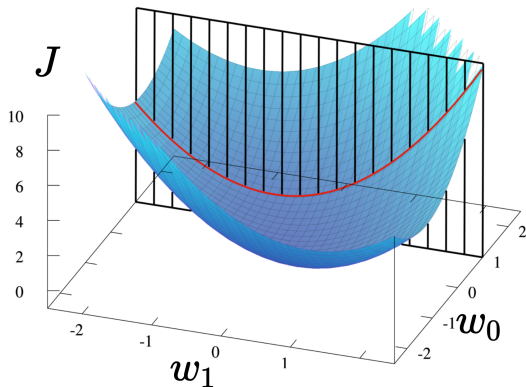
- Constrained optimization
- Non-smooth error

Gradients

For a cost function involving multiple variables, gradient is the partial derivative of the function w.r.t. each variable while fixing other variable.

For example, in a two-variable regression, the gradient of the coefficient w_1 is:

$$\frac{\partial}{\partial w_1} J(w_0, w_1) = \lim_{\epsilon \rightarrow 0} \frac{J(w_0, w_1 + \epsilon) - J(w_0, w_1 - \epsilon)}{2\epsilon}$$



Gradients

In a more general D variables setting, we have

$$\frac{\partial}{\partial w_d} J(w_0, w_1, \dots, w_{D-1}) = \lim_{\epsilon \rightarrow 0} \frac{J(w_d + \epsilon, \mathbf{w}_{\setminus d}) - J(w_d - \epsilon, \mathbf{w}_{\setminus d})}{2\epsilon}$$

where $\mathbf{w}_{\setminus d}$ denotes all coefficients except for coefficient w_d .

Gradients in matrix notation:

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial}{\partial w_0} J(\mathbf{w}) \\ \frac{\partial}{\partial w_1} J(\mathbf{w}) \\ \vdots \\ \frac{\partial}{\partial w_{D-1}} J(\mathbf{w}) \end{bmatrix}$$

General gradient descent algorithm

Gradient descent is an iterative algorithm for optimization.

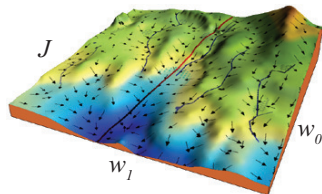
Algorithm 1 GradientDecent($\alpha = 0.005$, Convergence Criteria)

- 1: Initialize coefficients $\mathbf{w}^{\{0\}}$
 - 2: **while** convergence criterion is not met **do** // *what criteria can you think of?*
 - 3: $\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} - \alpha \nabla J(\mathbf{w}^{\{t\}})$ // *update using gradient*
 - 4: **end while**
-

where

- $\mathbf{w}^{\{t\}}$: weights at the t^{th} iteration
- α : learning rate
- $\nabla J(\mathbf{w}) =$
 $\left[\frac{\partial}{\partial w_0} J(\mathbf{w}), \frac{\partial}{\partial w_1} J(\mathbf{w}), \dots, \frac{\partial}{\partial w_{D-1}} J(\mathbf{w}) \right]^T$
are the gradients

Steepest descent into a *global or local optimum* of the error surface



Outline

Objectives

General gradient descent algorithm

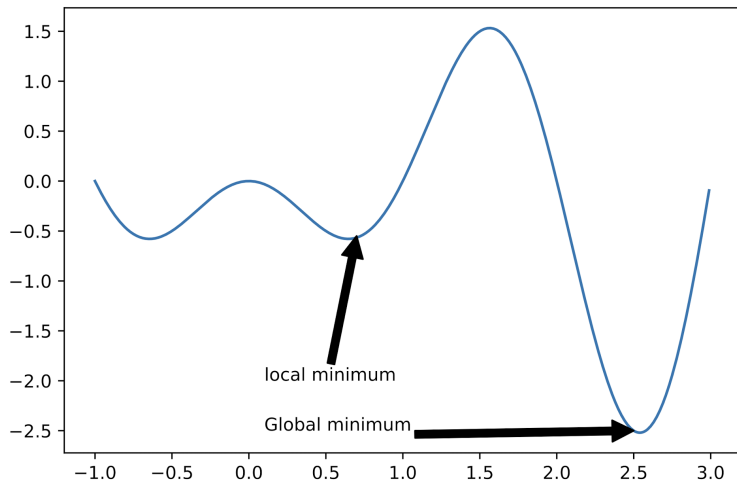
Function convexity

Choice of learning rate

Stochastic Gradient Descent (SGD)

Summary

How to determine if we can always find *the global optimum* not local optimum given a loss function?

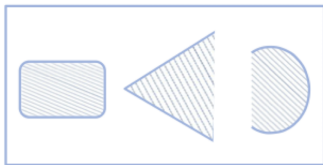


Convex sets

$\mathcal{S}$ is a **convex sets** if, for any $\mathbf{x}, \mathbf{x}' \in \mathcal{S}$, we have

$$\lambda \mathbf{x} + (1 - \lambda) \mathbf{x}' \in \mathcal{S}, \forall \lambda \in [0, 1]$$

In English: if we draw a line from $\mathbf{x}$ to $\mathbf{x}'$, all points on the line lie inside the convex set.



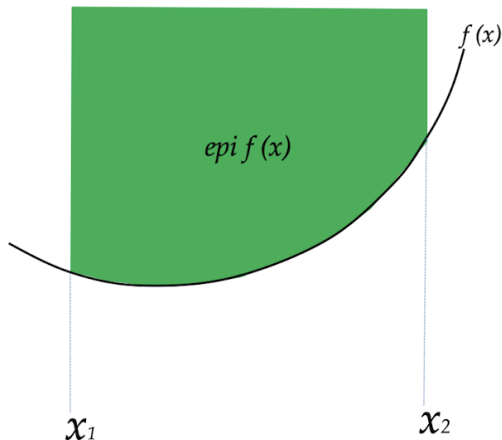
Convex



Not Convex

Convex functions

- *epigraph* is defined by a set of points above the function
- A function $f(\mathbf{x})$ is *convex* if its epigraph is a convex set

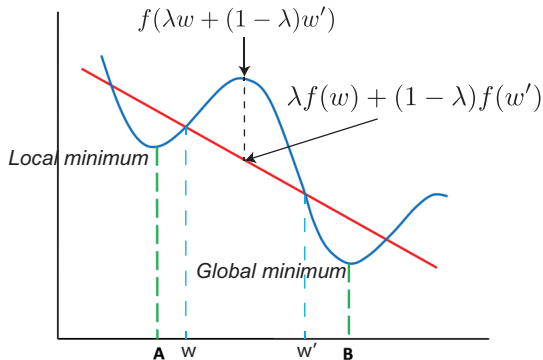
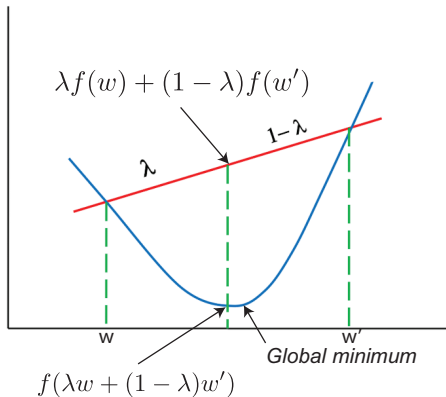


Examples of a convex and a non-convex function

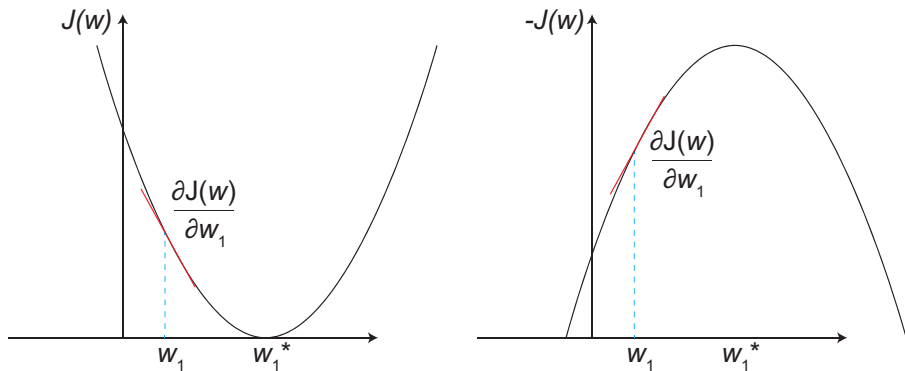
Equivalently, a function $f(w)$ is convex if

$$f(\lambda w + (1 - \lambda)w') \leq \lambda f(w) + (1 - \lambda)f(w')$$

where $f(\lambda w + (1 - \lambda)w')$ are points on the curve and $\lambda f(w) + (1 - \lambda)f(w')$ are points on the line. For example,



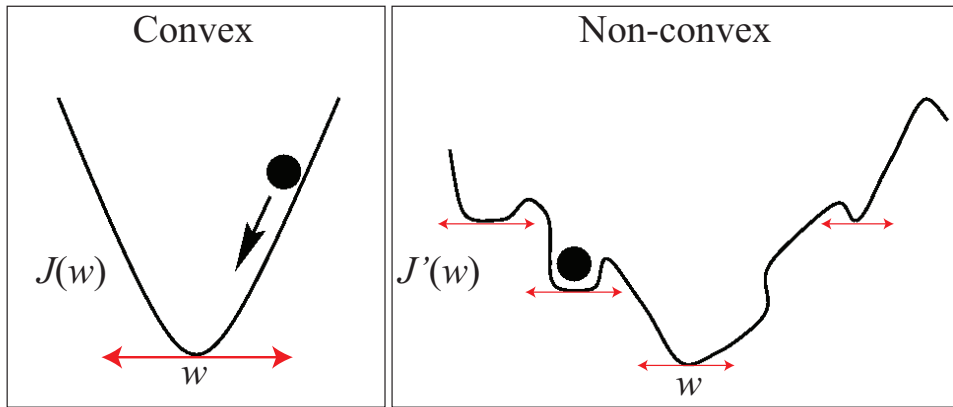
Negative of a convex function is a concave function



- For example, the SSE is a convex function and its negative function is a concave function and is proportional to the log Gaussian likelihood (Module 4.1).
- We subtract gradient from w_1 to minimize the loss, e.g., in the left figure, we subtract negative gradient from w to make it bigger. What's the sign of ∇w_1 when $w_1 > w_1^*$?
- We add gradient from w to maximize the log likelihood, e.g., in the right figure, we add positive gradient from w to make it bigger. What's the sign of ∇w_1 when $w_1 > w_1^*$?

Why do we care about convexity?

- Gradient descent (GD) can find the global minimum on a convex function.
- GD on a non-convex function may find local minimum (i.e., ultimately resulting in low prediction accuracy)



Testing function convexity by the second-order derivative (colab)

A convex function has an *increasing* first-order derivative, making it appear to bend upwards. A principled way to test whether a function $f(x)$ is convex is by taking the second order derivative of the function (i.e., differentiating the function twice w.r.t. x):

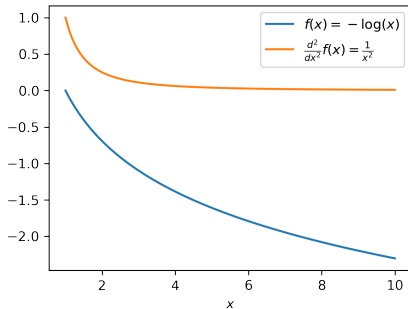
$$\frac{d}{dx} \frac{df(x)}{dx} = \frac{d^2 f(x)}{dx^2} \equiv \nabla^2 f(x)$$

For example, the first-order derivative of the negative natural log function $f(x) = -\log(x)$ is:

$$\frac{df(x)}{dx} = -\frac{1}{x}$$

and its second-order derivative is:

$$\frac{d^2 f(x)}{dx^2} = \frac{1}{x^2} > 0 \quad \forall x \in \mathbb{R}$$

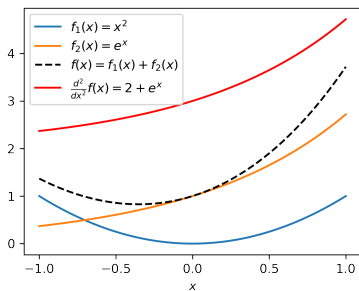


Some more examples of 1d convex functions $f(x)$: $-\sqrt{x}$, x^2 , e^x , $x \log(x)$

Sum and max of convex functions are still convex functions

Sum of convex functions is convex.

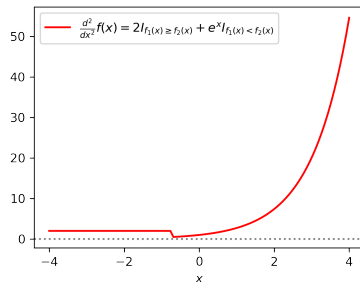
- Example 1. $f_1(x) = x^2$ and $f_2(x) = e^x$ are both convex and $f(x) = f_1(x) + f_2(x)$ is also convex.



- Example 2. sum of squared errors is a convex function:
$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \sum_n (\mathbf{x}^{(n)}\mathbf{w} - y^{(n)})^2$$

Maximum of convex functions is convex.

- Example 1. $f_1(x) = x^2$ and $f_2(x) = e^x$ are convex, $f(x) = \max(f_1(x), f_2(x))$ is also convex.



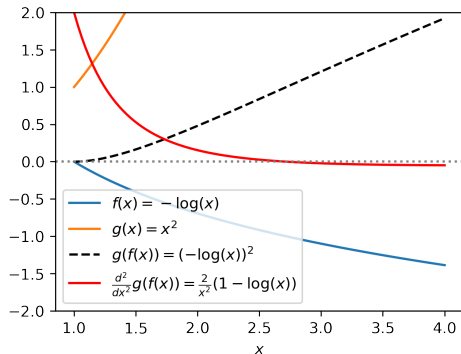
- Example 2. $f_1(x) = x^4$ and $f_2(x) = x \log(x)$ are convex, $f(x) = \max(f_1(x), f_2(x))$ is also convex.

Composition of convex functions may or may not be convex

For example, $f(x) = -\log(x)$ and $g(x) = x^2$ are convex but not:

$$g(f(x)) = (-\log(x))^2$$

$$\nabla^2 g(f(x)) = \frac{2}{x^2}(1 - \log(x)) < 0 \text{ if } x > e$$

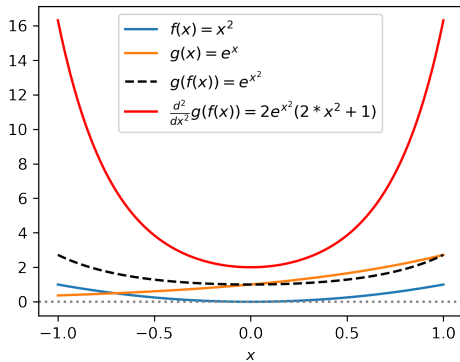


If $f(x)$, $g(x)$ are convex, and $g(x)$ is **non-decreasing**, then $g(f(x))$ is convex.

e.g.,

$$f(x) = x^2, g(x) = e^x, g(f(x)) = e^{x^2}$$

$$\nabla^2 g(f(x)) = 2e^{x^2}(x^2 + 1) > 0 \forall x \in \mathbb{R}$$



Is cross-entropy as the cost function for logistic regression convex?

$$J(\mathbf{w}) = \sum_{n=1}^N y^{(n)} \underbrace{\log(1 + \exp(-\mathbf{x}^{(n)}\mathbf{w}))}_{f_1(w)} + (1 - y^{(n)}) \underbrace{\log(1 + \exp(\mathbf{x}^{(n)}\mathbf{w}))}_{f_2(w)}$$

It is not obvious whether the composition functions are convex. Let's take their second-order derivatives. To simplify the notation, let's denote $z = wx$ for one input feature and one data point. We can show that both are convex by using chain-rule:

$$\frac{d^2 f_1(w)}{(dz)^2} = \frac{d}{dz} \frac{d \log(1 + \exp(-z))}{dz} = \frac{d}{dz} \frac{-\exp(-z)}{1 + \exp(-z)} = \frac{d}{dz} \frac{-1}{\exp(z) + 1} = \frac{\exp(z)}{(\exp(z) + 1)^2}$$

$$\frac{d^2 f_1(w)}{(dw)^2} = \frac{d^2 f_1(w)}{(dz)^2} \left(\frac{dz}{dw}\right)^2 = \frac{\exp(z)x^2}{(\exp(z) + 1)^2} \geq 0$$

$$\frac{d^2 f_2(w)}{(dz)^2} = \frac{d}{dz} \frac{d \log(1 + \exp(z))}{dz} = \frac{d}{dz} \frac{\exp(z)}{1 + \exp(z)} = \frac{d}{dz} \frac{1}{\exp(-z) + 1} = \frac{\exp(-z)}{(\exp(-z) + 1)^2}$$

$$\frac{d}{dw} \frac{df_2(w)}{dw} = \frac{d^2 f_2(w)}{(dz)^2} \left(\frac{dz}{dw}\right)^2 = \frac{\exp(-z)x^2}{(\exp(-z) + 1)^2} \geq 0$$

Cross-entropy as the cost function for logistic regression is *convex*

$$J(\mathbf{w}) = \sum_{n=1}^N y^{(n)} \underbrace{\log(1 + \exp(-\mathbf{x}^{(n)}\mathbf{w}))}_{f_1(w)} + (1 - y^{(n)}) \underbrace{\log(1 + \exp(\mathbf{x}^{(n)}\mathbf{w}))}_{f_2(w)}$$

- Based on the second order derivatives, both $f_1(w)$ and $f_2(w)$ are convex
- Because $y^{(n)} \in \{0, 1\}$, $y^{(n)} \frac{d^2 f_1(w)}{(dz)^2} \geq 0$ and $(1 - y^{(n)}) \frac{d^2 f_2(w)}{(dz)^2} \geq 0$.
- The sum of convex functions is convex.
- Therefore, the cross entropy function w.r.t. w is **convex**

Outline

Objectives

General gradient descent algorithm

Function convexity

Choice of learning rate

Stochastic Gradient Descent (SGD)

Summary

Gradient is less expensive than the closed-form solution in linear regression

- Recall for both linear regression with MSE loss $J(\mathbf{w}) = \frac{1}{2N} \sum_n (y^{(n)} - \hat{y}^{(n)})^2$ and logistic regression with MCE loss $J(\mathbf{w}) = \frac{1}{N} \sum_n -y^{(n)} \log(\hat{y}^{(n)}) - (1 - y^{(n)}) \log(1 - \hat{y}^{(n)})$, the gradients of the linear coefficients follow the same form (slide 24 in linear regression lecture and slide 13 in logistic regression lecture):

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_{n=1}^N (\hat{y}^{(n)} - y^{(n)}) \mathbf{x}^{(n)} = \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y}) / N$$

where $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ for linear regression and $\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w})$ for logistic regression.

- The time complexity is $O(ND)$ to compute this gradient.
- In contrast, the closed-form solution for the linear regression is: $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ which has time complexity is $O(ND^2 + D^3)$ (slide 26 in linear regression lecture).

```
1 def gradient(x, y, w):  
2     return np.dot(x.T, logistic(np.dot(x,w)) - y)/N
```

Gradient descent is easy to implement

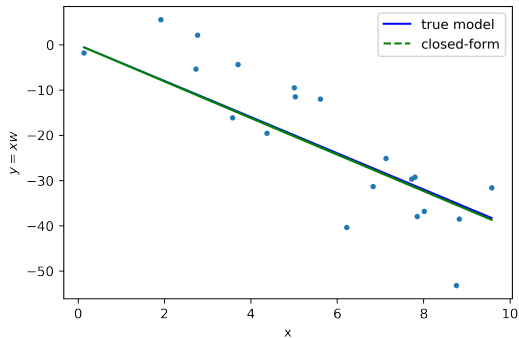
```
1 def gradient(x, y, w):  
2     return np.dot(x.T, logistic(np.dot(x,w)) - y)/N
```

```
1 def GradientDescent(x, # N x D  
2 y, # N  
3 lr=0.01, # learning rate  
4 eps=1e-2): # stop condition  
5     N, D = x.shape  
6     w = np.zeros(D)  
7     g = gradient(x, y, w)  
8     while np.linalg.norm(g) >  
9         ↪ eps:  
10         w = w - lr * g  
11         g = gradient(x, y, w)  
12     return w
```

Stopping criteria:

1. small gradient (as on the left)
2. maximum iterations
3. small change in cost function
4. increasing validation error to avoid overfitting

Gradient descent for linear regression



- Let's simulate a toy data with the slope of -4 and bias of 0:

$$y = -4x + e, \quad e \sim 10\mathcal{N}(0, 1)$$

- Cost: $L = \sum_{n=1}^{N=20} (y^{(n)} - x^{(n)}w)^2$
- First, let's fit a linear regression model using closed-form solution

$$\begin{aligned}\hat{w} &= (\mathbf{x}^\top \mathbf{x})^{-1} \mathbf{x}^\top \mathbf{y} = \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} = \frac{\sum_n x^{(n)} y^{(n)}}{\sum_n (x^{(n)})^2} \\ &= -4.04\end{aligned}$$

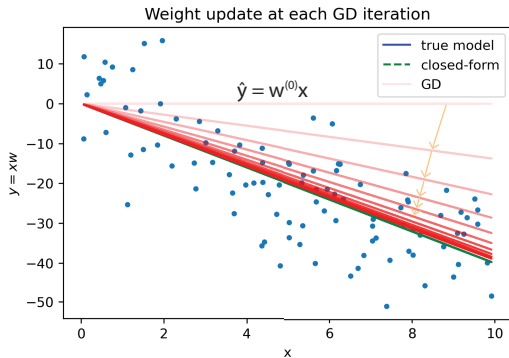
The closed-form estimate is very close to the true model ($w = -4$) such that the true line and fitted line on the plot almost coincide perfectly.

Gradient descent for linear regression

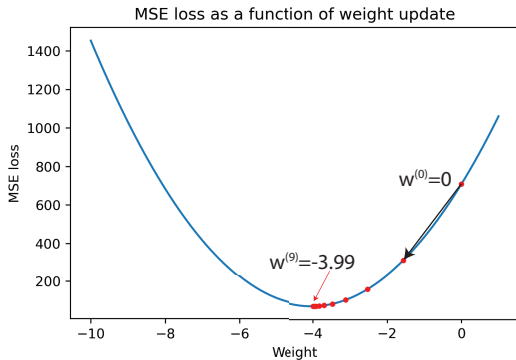
Fitting the model using gradient descent starting with $w^{(0)} = 0$ for 10 steps leads to an estimate of -3.99.

- 1: **for** $t \in \{0 \dots, 9\}$ **do**
- 2: $w^{\{t+1\}} \leftarrow w^{\{t\}} - 0.01 \nabla J(w^{\{t\}})$
- 3: **end for**

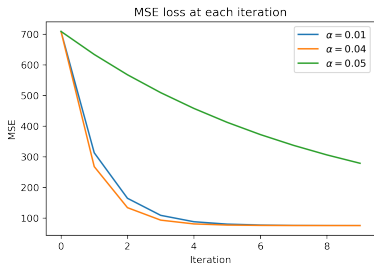
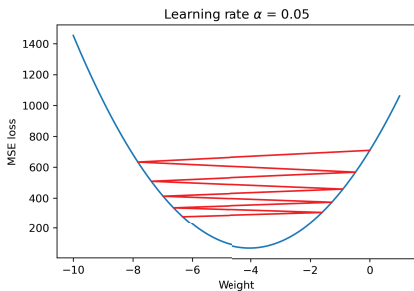
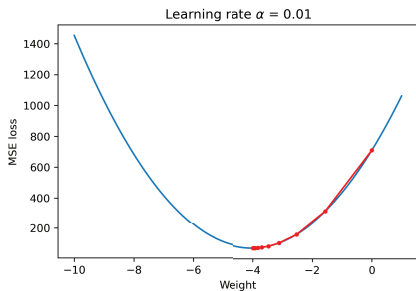
Data space



Cost function



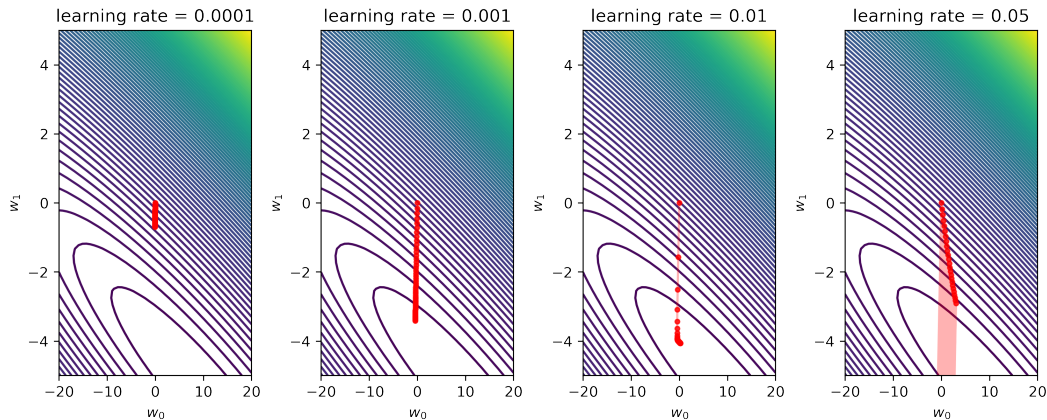
Impact of learning rates (α) on GD behaviour in 1D weight space



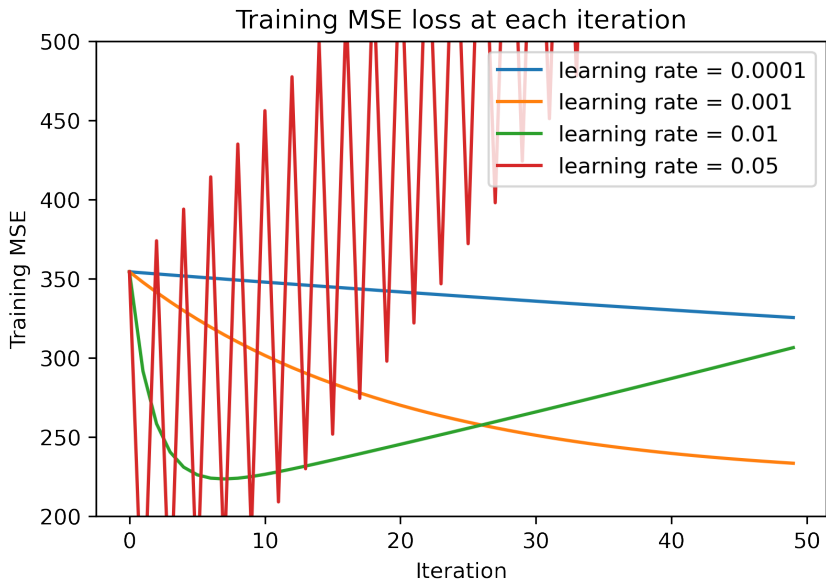
- On the one hand, having large learning rate (i.e., 0.05 in this example) will overshoot the optimal solution
- On the other hand, if the learning rate is too small, it will takes a long time to converge.

Impact of learning rates (α) on GD behaviour in 2D weight space (w_0, w_1)

Minimizing $L = \frac{1}{2N} \sum_{n=1}^{N=20} (y^{(n)} - w_0 - x^{(n)} w_1)^2$ w.r.t. w_0 and w_1 by GD.



Choosing a right learning rate based on the behaviour of error function



Outline

Objectives

General gradient descent algorithm

Function convexity

Choice of learning rate

Stochastic Gradient Descent (SGD)

Summary

Overall gradient is the sum of gradients over each training examples

The overall gradient can be written as the sum of individual gradients over the N training examples:

$$\begin{aligned}\frac{\partial J(\mathbf{w}; \mathcal{D})}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \left(\frac{1}{N} \sum_{n=1}^N (y^{(n)} - \mathbf{x}^{(n)} \mathbf{w})^2 \right) \\ &= \frac{1}{N} \sum_{n=1}^N \frac{\partial}{\partial \mathbf{w}} (y^{(n)} - \mathbf{x}^{(n)} \mathbf{w})^2 \\ &\equiv \frac{1}{N} \sum_{n=1}^N \frac{\partial J_n(\mathbf{w})}{\partial \mathbf{w}}\end{aligned}$$

Therefore, the overall gradient can be interpreted as the *expected* gradient over individual training examples:

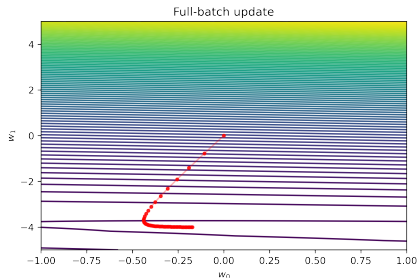
$$\frac{\partial J(\mathbf{w}; \mathcal{D})}{\partial \mathbf{w}} = \mathbb{E}_{\mathcal{D}} \left[\frac{\partial J_n(\mathbf{w})}{\partial \mathbf{w}} \right] = \mathbb{E}_{\mathcal{D}} [\nabla J_n(\mathbf{w})]$$

“Online” SGD update versus batch gradient update (Colab)

Full-batch training

We can update our weights after seeing *ALL* training examples:

$$\mathbf{w}^{\{t\}} \leftarrow \mathbf{w}^{\{t-1\}} - \alpha \nabla J(\mathbf{w}^{\{t-1\}})$$

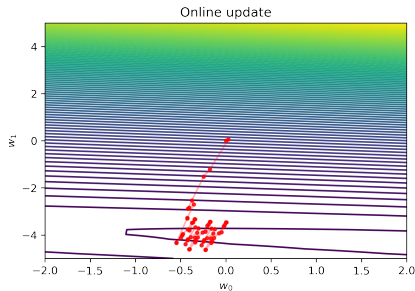


Online training

We can update our weights *right after* seeing *EACH* training example:

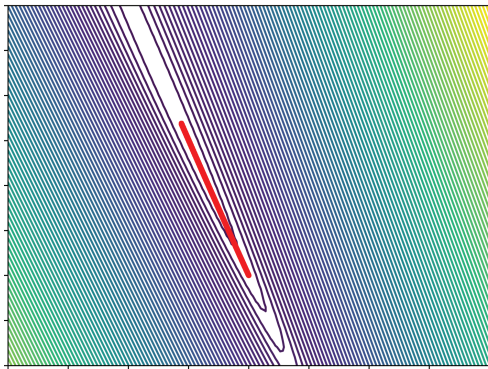
$$\mathbf{w}^{\{t\}} \leftarrow \mathbf{w}^{\{t-1\}} - \alpha \nabla J_n(\mathbf{w}^{\{t-1\}})$$

Each update is based only on one training example ($O(D)$) Each step is “on average” heading in the right direction.

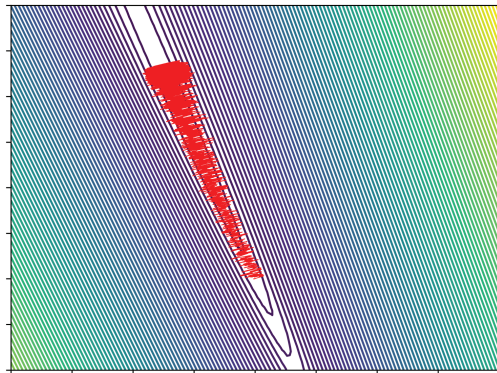


Full-batch update versus Online SGD on the Iris dataset

Full-batch gradient



Stochastic gradient



For SGD, the weights oscillate even at the global minimum.

Convergence of Online SGD

- Stochastic gradients are not zero even at the optimal $\mathbf{w}$.
- How to guarantee convergence?
- Idea: **schedule** to have decreasing learning rates over time

Robbins-Monro schedule:

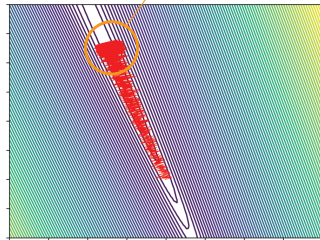
1. To reach the minimum, the sequence we use should satisfy: $\sum_{t=0}^{\infty} \alpha^{\{t\}} = \infty$
2. The learning rate should go to zero such that $\sum_{t=0}^{\infty} (\alpha^{\{t\}})^2 < \infty$

Scheduling examples:

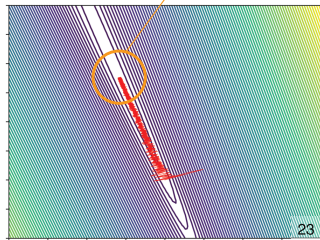
$$\alpha^{\{t\}} = \frac{10}{t}, \quad \alpha^{\{t\}} = t^{-0.51}$$

$\frac{10}{t}$ will not work at practice bc the initial learning rate is too large

Oscillate even at the global minimum



Reduce oscillation by smaller learning rate



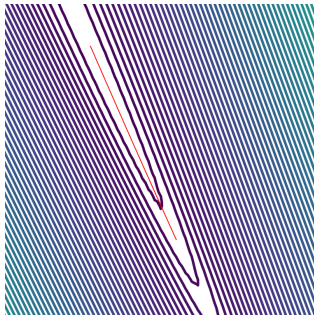
Minibatch SGD

Minibatch SGD strikes a balance between online SGD and full-batch gradient update by summing the gradients over the examples within the minibatch $\mathcal{B}$:

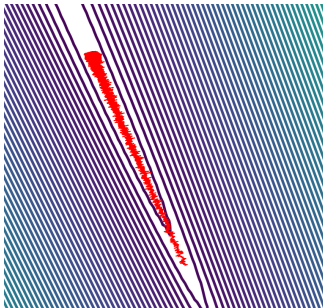
$$\nabla J_{\mathcal{B}} = \sum_{n \in \mathcal{B}} \nabla J_n(\mathbf{w})$$

where $\mathcal{B} \subseteq \{1, \dots, N\}$ is a subset of the training data.

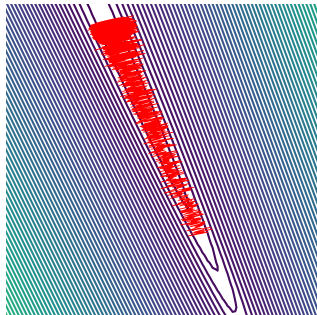
GD full batch



SGD minibatch-size=16

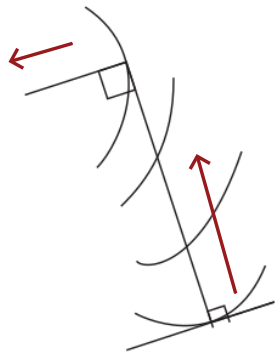
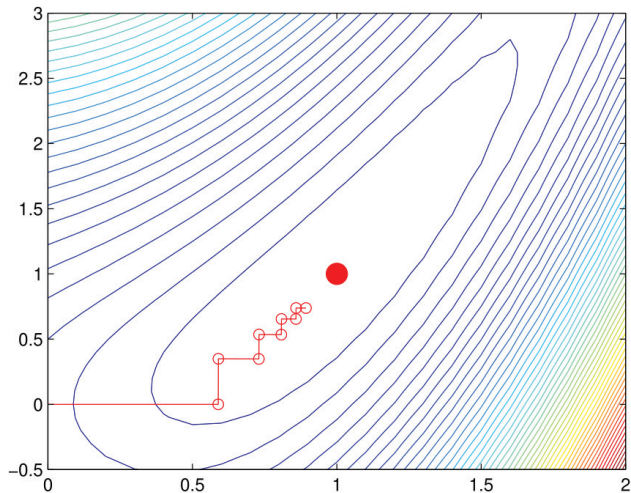


SGD minibatch-size=1



Oscillation behaviour of GD

GD can oscillate a lot. Each gradient step is perpendicular to the isocountours. In SGD, oscillation is worsened due to the noisy gradient estimate.



Momentum

Momentum mitigates the oscillations by having a running average of the gradients:

$$\begin{aligned}\nabla w^{\{t\}} &\leftarrow \beta \nabla w^{\{t-1\}} + (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{t-1\}}) \\ w^{\{t\}} &\leftarrow w^{\{t-1\}} - \alpha \nabla w^{\{t\}}\end{aligned}$$

where β controls the momentum: $\beta = 0$ reduces to standard SGD and $\beta = 1$ fixes the gradient. A common value for $\beta = 0.1$.

This is effectively an *exponentially moving* average:

$$\nabla w^{\{T\}} = \sum_{t=1}^{T-1} \beta^{T-t-1} (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{t\}})$$

Let's see how it works for the first $T = 3$ updates:

$$\begin{aligned}\nabla w^{\{1\}} &\leftarrow 0 \\ \nabla w^{\{2\}} &\leftarrow \beta \nabla w^{\{1\}} + (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{1\}}) \\ \nabla w^{\{3\}} &\leftarrow \beta \nabla w^{\{2\}} + (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{2\}}) \\ &= \beta \left(\beta \nabla w^{\{1\}} + (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{1\}}) \right) \\ &\quad + (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{2\}}) \\ &= \beta^1 (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{1\}}) \\ &\quad + \beta^0 (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{2\}}) \\ &= \sum_{t=1}^{T-1} \beta^{T-t-1} (1 - \beta) \nabla J_{\mathcal{B}}(w^{\{t\}})\end{aligned}$$

Momentum

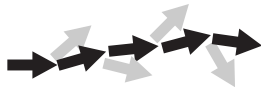
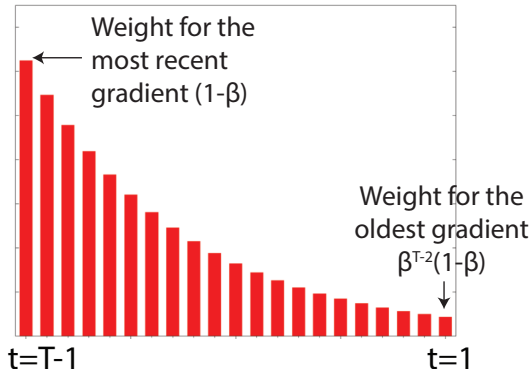
Momentum mitigates the oscillations by having a running average of the gradients:

$$\begin{aligned}\nabla_w\{t\} &\leftarrow \beta \nabla_w\{t-1\} + (1 - \beta) \nabla J_{\mathcal{B}}(w\{t-1\}) \\ w\{t\} &\leftarrow w\{t-1\} - \alpha \nabla_w\{t\}\end{aligned}$$

where β controls the momentum: $\beta = 0$ reduces to standard SGD and $\beta = 1$ fixes the gradient. A common value for $\beta = 0.1$.

This is effectively an *exponentially moving average*:

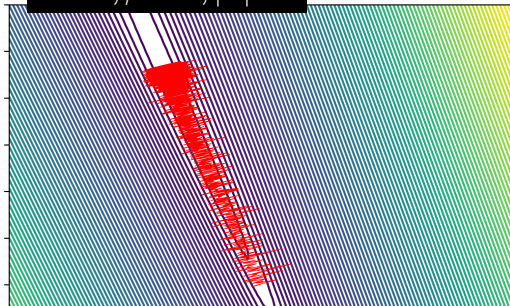
$$\nabla_w\{T\} = \sum_{t=1}^{T-1} \beta^{T-t-1} (1 - \beta) \nabla J_{\mathcal{B}}(w\{t\})$$



Momentum helps reduce oscillation

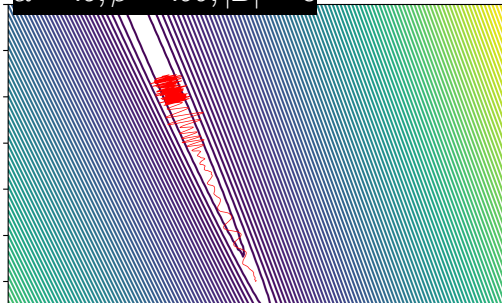
no momentum

$\alpha = .5, \beta = 0, |\mathbb{B}| = 8$



with momentum

$\alpha = .5, \beta = .99, |\mathbb{B}| = 8$



See more details on momentum here: <https://distill.pub/2017/momentum/>

Adaptive gradient (Adagrad)

Each parameter w_d may update at different rates (e.g., sparse data when some features are often zeros when using SGD).

We can use *adaptive* learning rate for *each individual* parameter w_d :

$$S_d^{\{t\}} \leftarrow S_d^{\{t-1\}} + \left(\frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}}) \right)^2$$

Learning rate is *inversely* weighted by the sum of squares of the derivatives over all iterations so far:

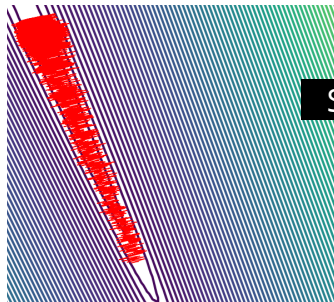
$$w_d^{\{t\}} \leftarrow w_d^{\{t-1\}} - \frac{\alpha}{\sqrt{S_d^{\{t\}} + \epsilon}} \frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}})$$

1. Learning rate is adapted to the accumulative updates
2. ϵ is used to avoid numerical issues

Adagrad's issue: for large gradient, the learning rate goes to zero too fast

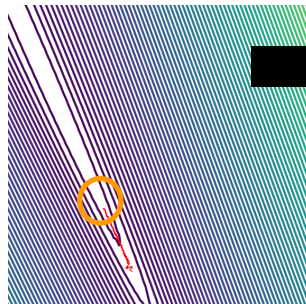
$$S_d^{\{t\}} \leftarrow S_d^{\{t-1\}} + \left(\frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}}) \right)^2$$
$$w_d^{\{t\}} \leftarrow w_d^{\{t-1\}} - \frac{\alpha}{\sqrt{S_d^{\{t\}} + \epsilon}} \frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}})$$

$\alpha = .1, |\mathbb{B}| = 1, T = 80,000$



SGD

$\alpha = .1, |\mathbb{B}| = 1, T = 80,000, \epsilon = 1e-8$



Adagrad

Fix to Adagrad's issue: Root Mean Squared Propagation (RMSprop)

Similar to the momentum idea, we can fix the problem of diminishing step-size with Adagrad by using an *exponential moving average* instead of sum:

$$S_d^{\{t\}} \leftarrow \gamma S_d^{\{t-1\}} + (1 - \gamma) \left(\frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}}) \right)^2$$
$$w_d^{\{t\}} \leftarrow w_d^{\{t-1\}} - \frac{\alpha}{\sqrt{S_d^{\{t\}} + \epsilon}} \frac{\partial}{\partial w_d} J(\mathbf{w}^{\{t-1\}}) \quad (\text{same as Adagrad})$$

In vector form:

$$\mathbf{S}^{\{t\}} \leftarrow \gamma \mathbf{S}^{\{t-1\}} + (1 - \gamma) \left(\nabla J(\mathbf{w}^{\{t-1\}}) \right)^2$$
$$\mathbf{w}^{\{t\}} \leftarrow \mathbf{w}^{\{t-1\}} - \frac{\alpha}{\sqrt{\mathbf{S}^{\{t\}} + \epsilon}} \nabla J(\mathbf{w}^{\{t-1\}})$$

where $\mathbf{S}^{\{t\}}$ is a vector and the squared root is an element-wise operation.

Putting it all together: Adam – Adaptive Moment Estimation

Adam combines momentum, Adagrad and RMSprop ideas:

$$\mathbf{M}^{\{t\}} \leftarrow \beta_1 \mathbf{M}^{\{t-1\}} + (1 - \beta_1) \nabla J(\mathbf{w}^{\{t-1\}}) \quad (\text{identical to momentum})$$

$$\mathbf{S}^{\{t\}} \leftarrow \beta_2 \mathbf{S}^{\{t-1\}} + (1 - \beta_2) \left(\nabla J(\mathbf{w}^{\{t-1\}}) \right)^2 \quad (\text{identical to RMSprop})$$

$$\hat{\mathbf{M}}^{\{t\}} \leftarrow \frac{\mathbf{M}^{\{t\}}}{1 - \beta_1^t}, \quad \hat{\mathbf{S}}^{\{t\}} \leftarrow \frac{\mathbf{S}^{\{t\}}}{1 - \beta_2^t}$$

$$\mathbf{w}^{\{t\}} \leftarrow \mathbf{w}_d^{\{t-1\}} - \frac{\alpha}{\sqrt{\hat{\mathbf{S}}^{\{t\}} + \epsilon}} \hat{\mathbf{M}}^{\{t\}} \quad (\text{inspired by Adagrad})$$

Let $\mathbf{M}^{\{0\}} = \mathbf{S}^{\{0\}} = 0$; at step $t = 1$

$$\mathbf{M}^{\{1\}} \leftarrow (1 - \beta_1) \nabla J(\mathbf{w}^{\{0\}}) \implies \hat{\mathbf{M}}^{\{1\}} \leftarrow \nabla J(\mathbf{w}^{\{0\}})$$

$$\begin{aligned} \mathbf{S}^{\{1\}} &\leftarrow (1 - \beta_2) \left(\nabla J(\mathbf{w}^{\{0\}}) \right)^2 \\ \implies \hat{\mathbf{S}}^{\{1\}} &\leftarrow \left(\nabla J(\mathbf{w}^{\{0\}}) \right)^2 \end{aligned}$$

$$\mathbf{w}^{\{1\}} \leftarrow \mathbf{w}^{\{0\}} - \frac{\alpha}{\sqrt{\hat{\mathbf{S}}^{\{1\}} + \epsilon}} \hat{\mathbf{M}}^{\{1\}}$$

- When t is small (but > 1), $\hat{\mathbf{M}}^{\{t\}}$ scales up $\mathbf{M}^{\{t\}}$; $\hat{\mathbf{S}}^{\{t\}}$ scales up $\mathbf{S}^{\{t\}}$.
- When t is large, $\beta^t \approx 0$, $\hat{\mathbf{M}}^{\{t\}} \approx \mathbf{M}^{\{t\}}$, $\hat{\mathbf{S}}^{\{t\}} \approx \mathbf{S}^{\{t\}}$
- Adam is the default optimization algorithm in practice (e.g., PyTorch)

Summary

- Learning: optimizing the model parameters (minimizing a cost function)
- Use gradient descent to find local minimum
 - Easy to implement
 - For convex functions, GD leads to global minimum
- Stochastic GD: for large data-sets use mini-batch for a noisy-fast estimate of gradient
 - Robbins-Monro condition: reduce the learning rate at increasing iteration to help stabilize convergence
- Momentum: exponential running average to focus SGD towards the (local) optimum
- Adagrad & RMSProp: per parameter adaptive learning rate
- Adam combines momentum, Adagrad, and RMSProp
- SGD are commonly applied in neural networks (Module 5).